

Multi-Agent Pac-Man — Test Report (Q1–Q3)

Date: 2026-09-27 | File under test: `multiAgents.py`

Note on scope: No `autograder.py` / `test_cases` were present in this copy of the project, so the official grading script could not be run. Everything below is a manual, from-scratch verification using `pacman.py` directly, run through `-c` (catch exceptions/timeouts) and `-q` (quiet, batch mode).

Question 1 — ReflexAgent (evaluationFunction)

Layout	Games	Win Rate	Avg Score
testClassic	5	5/5 (100%)	561.2
openClassic	10	10/10 (100%)	1240.4
mediumClassic	10	5/10 (50%)	873.7

Assessment: Correct and functional. Perfect play on the easy/open layouts. The 50% win rate on `mediumClassic` is in the normal range for a reflex (one-ply, no-lookahead) heuristic — it has no way to see a ghost trap coming more than one move ahead, so occasional deaths there are expected, not a bug.

Question 2 — MinimaxAgent (getAction)

Layout	Depth	Games	Win Rate	Avg Score
testClassic	2	3	3/3 (100%)	470.7
minimaxClassic	4	10	5/10 (50%)	11.8
trappedClassic	3	5	0/5 (0%)	-501.0

Assessment: Correct.

- `minimaxClassic` uses a `RandomGhost`, not an adversarial one. Minimax assumes the worst case, so it isn't tuned to exploit random play — a ~50% win rate here is the textbook expected result for this layout/question, not a defect.
- `trappedClassic` is specifically designed so that a correctly-implemented minimax agent recognizes it cannot win no matter what it does, and instead minimizes damage (dies immediately rather than stalling). A 0% win rate with a consistent -501 score is the correct expected output for this layout — it demonstrates the agent is genuinely evaluating the game tree, not just the heuristic evaluation function.
- One isolated run threw *"Agent 0 ran out of time!"* on `testClassic` (depth=2) during a batch of 3 games. This did not reproduce across 8 additional single-game runs plus 3 repeats of the exact original command — all fast (under 2s) and correct. Given `testClassic` has only 2 agents and depth=2, the search tree is tiny (bounded regardless of maze state), so this looks like a one-off system/OS scheduling hiccup, not an algorithmic bug.

Question 3 — AlphaBetaAgent (getAction)

Layout	Depth	Games	Win Rate	Avg Score
--------	-------	-------	----------	-----------

testClassic	2	3	3/3 (100%)	517.3
minimaxClassic	4	10	5/10 (50%)	11.0
trappedClassic	3	5	0/5 (0%)	-501.0

Matches MinimaxAgent's results on every layout, as expected (alpha-beta pruning must never change which action is chosen — only how much work it takes to find it).

Correctness check: identical decisions under a fixed random seed

Same layout, same depth, same fixed seed (-f) so both agents face the exact same ghost moves:

Agent	Scores (5 games)	Win rate
MinimaxAgent	516, 516, 516, 516, -492	4/5
AlphaBetaAgent	516, 516, 516, 516, -492	4/5

Identical, game-for-game. This is the strongest evidence that the alpha-beta implementation is behaviorally equivalent to full minimax.

Performance check: pruning actually saves work

Same fixed-seed single game on smallClassic, depth=3, 3 repeats per agent for stability:

Agent	Score (every run)	Wall time (avg of 3 runs)
MinimaxAgent	1530	7.77s
AlphaBetaAgent	1530	4.54s

Same final score every time, ~42% faster with pruning. This confirms alpha-beta is doing real work — cutting off branches — without altering the outcome.

Note: an earlier, unseeded timing comparison in this session showed noisier/contradictory numbers because different random ghost moves between runs produce different-shaped search trees. The fixed-seed test above is the reliable one — always compare timings with -f for apples-to-apples results.

Summary

Question	Status	Key evidence
Q1 ReflexAgent	Working	100% win rate on testClassic/openClassic
Q2 MinimaxAgent	Working	Correct worst-case behavior on minimaxClassic and trappedClassic
Q3 AlphaBetaAgent	Working	Identical decisions to Minimax (fixed seed) + ~42% speedup

No crashes or exceptions in any test run except the single unreproduced timeout noted under Q2, which is not believed to indicate a bug.